ASSIGNMENT 5 – CO5

DATA ACQUISITION, CLEANING, ANALYSIS AND AI APPLICATIONS

Course: Python Programming (CSA0818)

Student Name: M SANTHOSH KUMAR

Register Number: 192412268

PART A – DATA AND PROGRAM DESIGN (25 Marks)

Task A1 – Design the Data Acquisition (7 Marks)

A1.1 Inspect the Source (2 Marks)

Expected CSV columns: station,date,hour,pm25,pm10,no2,temperature. Example: Adyar,2026-10-01,08,48,72,18,31.5.

Quality problem	Example	Treatment / risk
Missing reading	blank or NA	Treat as missing, not zero; apply documented imputation.
Negative pollutant	pm25 = -5	Flag as sensor fault and exclude from valid statistics.
Impossibly large value	pm25 = 1500	Flag as invalid; use a documented threshold, such as >999 for the supplied test.
Mixed temperature units	Fahrenheit from two stations	Convert F to C using (F−32)×5/9.
Duplicate observation	same station/date/hour twice	Deduplicate using a documented key and policy.
Malformed date/hour	2026/31/10 or hour 27	Parse and validate before storing.
Inconsistent station name	'adyar' and 'Adyar '	Strip spaces and map to canonical station IDs.
Non-numeric reading	'high'	Catch conversion errors and log the row.
Missing columns/wrong delimiter	pm25 absent or semicolon-separated	Validate header/schema before processing.
Wrong encoding/empty file	invalid UTF-8 or no rows	Handle as file-level errors; do not silently accept.

A1.2 Design the Storage (3 Marks)

```
CREATE TABLE stations (
    station_id INTEGER PRIMARY KEY,
    station_name TEXT NOT NULL UNIQUE,
    temperature_unit TEXT NOT NULL DEFAULT 'C'
        CHECK (temperature_unit IN ('C','F'))
);

CREATE TABLE readings (
    station_id INTEGER NOT NULL,
    reading_date TEXT NOT NULL,
    hour INTEGER NOT NULL CHECK (hour BETWEEN 0 AND 23),
    pm25 REAL, pm10 REAL, no2 REAL, temperature_c REAL,
    source_file TEXT,
    PRIMARY KEY (station_id, reading_date, hour),
    FOREIGN KEY (station_id) REFERENCES stations(station_id)
);
CREATE INDEX idx_readings_station_date
ON readings(station_id, reading_date);
```

Map station names to station_id; normalise dates to YYYY-MM-DD and hour to integer 0–23; convert pollutant fields to numeric values or NULL; convert all temperatures to Celsius before insertion. The composite primary key prevents duplicate hourly records.

A1.3 CSV or SQL? (2 Marks)

Question	CSV/DataFrame	SQLite
Adyar average PM2.5 in October	Read and filter files; repeated questions scan the data.	SELECT AVG(pm25) FROM readings JOIN stations USING(station_id) WHERE

		station_name=? AND reading_date BETWEEN ? AND ?;
Worst hour by mean PM2.5	Load and group by hour each time.	GROUP BY hour and ORDER BY AVG(pm25) DESC.
Compare station monthly means	Read and group all relevant CSV rows.	GROUP BY station with a date filter and suitable index.

Loading all CSV data for each question is typically O(n) time and O(n) memory for a full DataFrame. SQLite has an initial import cost and stores data on disk; an indexed selective query can be approximately O(log n + k), where k is the number of matching rows. Actual performance depends on the query plan and selectivity.

Task A2 – Design the Cleaning and Normalisation (5 Marks)

A2.1 Cleaning Rules (2 Marks)

Problem	Rule and justification
Missing values	Convert blank/NA to missing. Forward-fill only short gaps where time-series context justifies it; otherwise use station/month median or leave missing. Never replace with zero automatically.
Impossible values	Mark negative pollutant values and values above a documented sensor threshold as missing and log them.
Duplicates	Deduplicate by station/date/hour with a documented first/last/quality-flag policy.
Fahrenheit	Convert using C = (F−32)×5/9 so all stations use one unit.

A2.2 Normalisation (2 Marks)

```
# Min-max normalisation
scaled = (x - minimum) / (maximum - minimum)

# Z-score standardisation
z = (x - mean) / standard_deviation
```

Min-max maps values to [0,1] when max > min; handle constant columns separately to avoid division by zero. It is useful when a bounded scale is required but is sensitive to outliers. Z-score centres values around zero with unit standard deviation and is useful for scale-sensitive methods, but is also affected by outliers. Fit parameters on reference/training data and reuse them consistently.

A2.3 Formatting (1 Mark)

- Dates use YYYY-MM-DD; hour is integer 0–23.
- Station names are stripped and mapped to canonical IDs.
- Measurements are numeric; missing values use SQL NULL/None, not the text 'NA'; temperature is stored in °C.

Alternative analysis: dropping missing rows and forward filling are O(n). Sorting to compute a median is O(n log n), though selection methods can be expected O(n). Dropping can bias means if gaps correlate with pollution peaks; forward filling can flatten changes; median imputation can hide real variation. Memory is O(n) if all readings are collected, lower for a streaming pass.

Task A3 – Design the Analysis and Visualisation (5 Marks)

A3.1 Choose the Charts (3 Marks)

Question	Chart and reason

PM2.5 trend over a month	Line chart: makes temporal rises and falls visible.
Compare stations	Bar chart of mean/median PM2.5 by station.
Temperature versus PM2.5	Scatter plot: shows association, clusters and outliers.
Hours with worst air	Bar chart of mean PM2.5 by hour; box plot can show spread.

A3.2 Daily Summary Table (2 Marks)

Group by station and date, reporting minimum, maximum, mean and count of valid PM2.5 readings. SQL example:

```
SELECT station_id, reading_date, MIN(pm25), MAX(pm25),
       AVG(pm25), COUNT(pm25)
FROM readings
GROUP BY station_id, reading_date;
```

A Python loop can maintain sum/count/min/max per group in one pass. Both SQL GROUP BY and one-pass Python are O(n) time. Loading all rows in Python costs O(n) memory; an incremental dictionary uses O(g) memory for g station-day groups. SQL may use temporary storage for grouping and returns only summary rows.

Task A4 – Plan Debugging and Profiling (4 Marks)

A4.1 Debugging Plan (2 Marks)

- Print small samples of raw, parsed and cleaned rows during initial debugging; remove or replace prints after diagnosis.
- Use assert for development invariants such as hour range and non-negative cleaned values; do not use assertions as input validation.
- Use logging levels: DEBUG for row details, INFO for counts, WARNING for rejected rows and ERROR for file/database failures.
- Use pdb or an IDE debugger to set breakpoints, inspect variables and step through the first invalid record.

A4.2 Profiling Plan (2 Marks)

```
from time import perf_counter
start = perf_counter()
# call the pipeline here
elapsed = perf_counter() - start
print(f"Elapsed: {elapsed:.6f} s")

# Microbenchmark:
# python -m timeit -s "from module import clean_one" "clean_one(sample)"

# Whole-program profile:
# python -m cProfile -s cumulative pipeline.py
```

Use perf_counter for end-to-end wall-clock timing, timeit for repeated isolated operations, and cProfile to find functions with high cumulative/self time. Repeat on representative inputs. Thousands of print calls can flood the console and slow processing. Logging can be filtered or redirected by level/handler; debugger overhead disappears when running normally. Measure diagnostics on/off rather than assuming a fixed overhead.

Task A5 – Design an AI Application for Citizen Complaints (4 Marks)

A5.1 Lexicon Sentiment and Extractive Summary (2 Marks)

```
import re
POSITIVE = {"good", "clean", "fast", "helpful", "improved"}
NEGATIVE = {"bad", "dirty", "slow", "rude", "smoke", "broken"}
NEGATORS = {"not", "never", "no"}

def score(text):
```

```
    words = text.lower().replace(".", " ").replace(",", " ").split()
    total = 0
    for i, word in enumerate(words):
        value = 1 if word in POSITIVE else -1 if word in NEGATIVE else 0
        if i > 0 and words[i - 1] in NEGATORS:
            value = -value
        total += value
    return total

def label(text):
    s = score(text)
    return "positive" if s > 0 else "negative" if s < 0 else "neutral"

def extractive_summary(text, k=2):
    sentences = [s.strip() for s in re.split(r'(?<=[.!?])\\s+', text) if s.strip()]
    stop = {"the","a","an","and","is","are","to","of","in","it","was"}
    words = re.findall(r"[a-z]+", text.lower())
    freq = {}
    for w in words:
        if w not in stop:
            freq[w] = freq.get(w, 0) + 1
    scored = []
    for i, sentence in enumerate(sentences):
        terms = re.findall(r"[a-z]+", sentence.lower())
        scored.append((sum(freq.get(w, 0) for w in terms if w not in stop), i, sentence))
    chosen = sorted(sorted(scored, reverse=True)[:k], key=lambda x: x[1])
    return " ".join(item[2] for item in chosen)
```

The lexicon scorer adds +1 for a positive word and −1 for a negative word, flipping the score when the immediately preceding token is a negator. The extractive summariser selects up to k sentences with the highest frequency-weighted important words and returns them in original order. It is a baseline, not a language-understanding system.

A5.2 Rule-Based Chatbot (2 Marks)

```
import re

def answer_question(con, question, today):
    q = question.lower()
    match = re.search(r"pm\\s*2?\\.?5\\s+in\\s+([a-z][a-z ]*?)\\s+(?:today|on)", q)
    if "pm2.5" not in q and "pm 2.5" not in q:
        return "I can answer questions about PM2.5 readings."
    if not match:
        return "Please ask: What is the PM2.5 in Adyar today?"
    station = match.group(1).strip().title()
    row = con.execute(
        "SELECT r.pm25 FROM readings r JOIN stations s USING(station_id) "
        "WHERE s.station_name=? AND r.reading_date=? "
        "ORDER BY r.hour DESC LIMIT 1", (station, today)
    ).fetchone()
    return (f"Latest PM2.5 for {station}: {row[0]}" if row
            else f"No valid reading found for {station} on {today}.")
```

Limitations: exact keywords and simple station extraction, no conversational memory, weak handling of misspellings or ambiguity, and no guarantee that one sensor reading represents a whole neighbourhood. Parameterised SQL protects against injection.

Alternative analysis: a trained Naive Bayes classifier may learn word combinations from labelled examples; a pre-trained language model may understand context better but uses more memory/compute. Lexicon scoring is O(w) per complaint with tiny fixed memory. Model inference depends on model size and input length. Evaluate on labelled local complaints, including code-mixing and sarcasm.

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION (25 Marks)

Task B1 – Implement and Validate the Cleaning Pipeline (7 Marks)

B1.1 Complete TODO 1 and TODO 2 (2 Marks)

```
import csv
import io

RAW = \"\"\"station,date,pm25
Adyar,2026-10-01,48
Adyar,2026-10-02,NA
Adyar,2026-10-03,
Guindy,2026-10-01,95
Guindy,2026-10-02,-5
Guindy,2026-10-03,121
\"\"\"

def load_clean(text):
    rows = []
    for row in csv.DictReader(io.StringIO(text)):
        raw = (row.get("pm25") or "").strip()
        if not raw or raw.upper() == "NA":
            continue
        try:
            pm = float(raw)
        except ValueError:
            continue
        if pm < 0 or pm > 999:
            continue
        station = (row.get("station") or "").strip()
        day = (row.get("date") or "").strip()
        if station and day:
            rows.append((station, day, pm))
    return rows

def min_max(values):
    if not values:
        return []
    lo, hi = min(values), max(values)
    if hi == lo:
        return [0.0 for _ in values]
    return [round((v-lo)/(hi-lo), 3) for v in values]

clean = load_clean(RAW)
print(clean)
print(min_max([pm for _, _, pm in clean]))
```

Expected output: [('Adyar', '2026-10-01', 48.0), ('Guindy', '2026-10-01', 95.0), ('Guindy', '2026-10-03', 121.0)] and [0.0, 0.644, 1.0]. The code also rejects values above 999. Test by adding Other,2026-10-04,1200 and confirming it is excluded.

B1.2 Compare with Design (1 Mark)

The code skips blank/NA, negative and over-999 PM2.5 values and converts accepted values to float. The supplied TODO data only contains station/date/PM2.5, so temperature conversion and PM10/NO2 checks cannot be demonstrated until the full dataset is provided. The 999 limit is a test rule, not a universal air-quality standard.

B1.3 Store and Query (2 Marks)

```
import sqlite3

readings = [("Adyar","2026-10-01",48.0), ("Guindy","2026-10-01",95.0),
            ("Guindy","2026-10-03",121.0), ("Adyar","2026-10-04",60.0)]
con = sqlite3.connect("air_quality.db")
```

```
con.execute("CREATE TABLE IF NOT EXISTS aq (station TEXT, day TEXT, pm25 REAL)")
con.executemany("INSERT INTO aq(station, day, pm25) VALUES (?, ?, ?)", readings)
con.commit()
print("Adyar October average:",
      con.execute("SELECT AVG(pm25) FROM aq WHERE station=? AND day BETWEEN ? AND ?",
                  ("Adyar","2026-10-01","2026-10-31")).fetchone())
print("Station means:", con.execute("SELECT station, AVG(pm25) FROM aq GROUP BY station").fetchall())
print("Daily means:", con.execute("SELECT day, AVG(pm25) FROM aq GROUP BY day").fetchall())
con.close()
```

For repeatable runs, clear/recreate the table or add a unique key; otherwise reruns may duplicate rows. Use placeholders for query values.

B1.4 Visualise (2 Marks)

```
# Requires pandas and matplotlib; assumes `clean` is available.
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame(clean, columns=["station", "date", "pm25"])
df["date"] = pd.to_datetime(df["date"])
daily = df.groupby(["date", "station"], as_index=False)["pm25"].mean()
for station, group in daily.groupby("station"):
    plt.plot(group["date"], group["pm25"], marker="o", label=station)
plt.xlabel("Date"); plt.ylabel("PM2.5")
plt.title("Daily PM2.5 Trend by Station"); plt.legend(); plt.tight_layout()
plt.savefig("pm25_trend.png"); plt.close()

df.groupby("station")["pm25"].mean().plot(kind="bar")
plt.xlabel("Station"); plt.ylabel("Mean PM2.5")
plt.title("Mean PM2.5 by Station"); plt.tight_layout()
plt.savefig("station_comparison.png"); plt.close()
```

Write interpretations after viewing your actual charts. The supplied TODO dataset has no temperature or hour columns, so it cannot support a temperature scatter plot or worst-hour chart. Do not invent conclusions from missing data.

Task B2 – Engineer Adversarial Data (5 Marks)

Complete TODO 1 and TODO 2

```
import sqlite3
readings = [("Adyar","2026-10-01",48.0), ("Guindy","2026-10-01",95.0),
            ("Guindy","2026-10-03",121.0), ("Adyar","2026-10-04",60.0)]
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE aq (station TEXT, day TEXT, pm25 REAL)")
con.executemany("INSERT INTO aq(station, day, pm25) VALUES (?, ?, ?)", readings)
con.commit()

def station_average(name):
    row = con.execute("SELECT AVG(pm25) FROM aq WHERE station = ?", (name,)).fetchone()
    return row[0]

print(station_average("Adyar"), station_average("Guindy"))
print(station_average("Adyar' OR '1'='1"))
con.close()
```

Expected output: 54.0 108.0, then None. The injection attempt is bound as a literal station name; it cannot alter SQL syntax.

Adversarial test plan

Input	Expected response	Lesson
Missing CSV column	Schema validation reports missing column.	Validate headers before reading rows.
Wrong delimiter	Header/row validation fails clearly.	Do not silently assume delimiters.

Invalid UTF-8	Catch UnicodeDecodeError and log file-level failure.	Encoding failure differs from a damaged row.
Empty file	Report no rows/missing header.	Empty input is not a valid full dataset.
Quote in station name	Parameterised query matches only literal value.	Binding prevents SQL injection.
Sarcasm/double negative	Lexicon can label incorrectly.	Word lists miss context and pragmatics.

If SQL is built with string concatenation, input such as Adyar' OR '1'='1 can alter the WHERE clause and may return multiple rows. A ? placeholder treats it as data. executemany and execute-in-a-loop both perform O(n) insertion work, but executemany reduces Python call overhead; use one transaction rather than committing each row.

Task B3 – Loops vs Libraries vs SQL (8 Marks)

B3.1 Three Methods and Correctness (2 Marks)

```
import csv, sqlite3, statistics, math
from collections import defaultdict

def means_with_csv(path):
    groups = defaultdict(list)
    with open(path, newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            groups[(row["station"], row["day"])].append(float(row["pm25"]))
    return {key: statistics.mean(vals) for key, vals in groups.items()}

def means_with_pandas(path):
    import pandas as pd
    df = pd.read_csv(path)
    return df.groupby(["station","day"])["pm25"].mean().to_dict()

def means_with_sql(path):
    con = sqlite3.connect(":memory:")
    con.execute("CREATE TABLE aq(station TEXT, day TEXT, pm25 REAL)")
    with open(path, newline="", encoding="utf-8") as f:
        rows = [(r["station"],r["day"],float(r["pm25"])) for r in csv.DictReader(f)]
    con.executemany("INSERT INTO aq VALUES (?,?,?)", rows)
    result = {(s,d):m for s,d,m in con.execute(
        "SELECT station,day,AVG(pm25) FROM aq GROUP BY station,day")}
    con.close()
    return result

# Compare keys and use math.isclose() for each mean.
# For a fair query-only SQL benchmark, populate the database before timing.
```

Verify that all three methods produce the same station-day keys and numerically close means. The SQL example includes reading/insertion time; measure query-only time separately when the database is already populated.

B3.2 Generate and Time Data (2 Marks)

```
import csv, time
from datetime import date, timedelta

def generate_csv(path, stations):
    start = date(2026, 1, 1)
    with open(path, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f); w.writerow(["station","day","pm25"])
        for s in range(stations):
            for d in range(365):
                day = start + timedelta(days=d)
                for hour in range(24):
                    value = float((s*7 + d*3 + hour) % 180)
```

```
                    w.writerow([f"Station{s+1}", day.isoformat(), value])

def timed(label, fn, *args):
    start = time.perf_counter(); result = fn(*args)
    print(label, f"{time.perf_counter()-start:.3f}s")
    return result

# Generate 1, 12 and 120 station files (365*24 = 8,760 rows/station).
# Run all methods on each file and save your actual times in a CSV.
```

Measure the same data and state whether loading/insertion is included. Record hardware, Python/library versions and repeat runs. Do not invent benchmark numbers.

B3.3 Profile (2 Marks)

```
# Save the loop method in loop_analysis.py, then run:
# python -m cProfile -s cumulative loop_analysis.py
#
# Or from Python:
import cProfile
# cProfile.run("means_with_csv('air_quality.csv')", sort="cumulative")
```

Name the slowest function from your actual cProfile output; the result depends on data size, machine, Python version and implementation.

B3.4 Engineering Trade-Off (2 Marks)

CSV loops have few dependencies and can stream data. Pandas offers concise vectorised operations but often holds the DataFrame in memory. SQL is good for persistent data, repeatable queries and indexed filtering. CSV/pandas can be simplest for a one-time analysis; SQLite is a practical choice for a daily report on one machine when concurrency and volume remain modest. Confirm with measurements.

A one-pass running sum/count dictionary is O(n) time and O(g) memory for g station-day groups. Storing every reading and then calling statistics.mean is O(n) time and O(n) memory. If raw readings must be retained, keep them in the database rather than only in an aggregation dictionary.

Task B4 – Final Engineering Decision (5 Marks)

B4.1 Sentiment and Summary Evaluation (2 Marks)

Run score(), label() and extractive_summary() on at least 50 complaints. Manually label each complaint, compare model labels with your labels and compute agreement = matching labels / 50 × 100%. Include disagreements and examples of sarcasm, mixed sentiment and double negatives. Do not claim an agreement score until you complete this review.

B4.2 Database Index (1 Mark)

```
con.execute("CREATE INDEX IF NOT EXISTS idx_aq_station_day ON aq(station, day)")
con.commit()
# Time the same selective query before/after index creation with perf_counter.
# Repeat many times and report the observed median/mean. Tiny tables may show no gain.
```

B4.3 Production Decision (2 Marks)

With 112 stations reporting every 15 minutes, the expected volume is 112 × 96 = 10,752 readings/day, approximately 3.93 million readings/year before overhead. Use validated ingestion, canonical units, SQL NULL for missing readings, indexes on station/time, rejected-row logs and raw-data retention. SQLite may be adequate for a single-machine workflow; consider a server database if concurrency, availability, retention or throughput needs exceed its practical limits.

Lexicon sentiment is cheap but weak on sarcasm, misspellings, code-mixing and nuanced complaints. Start with the baseline, evaluate it on a labelled sample, and adopt a trained model only if measured precision/recall or agreement is inadequate. Use your measured index timings and complaint agreement in the final decision.

Complete the Code – Sentiment TODO

```
POSITIVE = {"good", "clean", "fast", "helpful", "improved"}
NEGATIVE = {"bad", "dirty", "slow", "rude", "smoke", "broken"}
NEGATORS = {"not", "never", "no"}

def score(text):
    words = text.lower().replace(".", " ").replace(",", " ").split()
    total = 0
    for i, word in enumerate(words):
        value = 1 if word in POSITIVE else -1 if word in NEGATIVE else 0
        if i > 0 and words[i-1] in NEGATORS:
            value = -value
        total += value
    return total

def label(text):
    s = score(text)
    return "positive" if s > 0 else "negative" if s < 0 else "neutral"

for c in ["Roads are clean and the bus was fast.",
          "Smoke from the factory, air is bad.",
          "The staff were not helpful.",
          "Not bad, service improved."]:
    print(f"{label(c):8} | {c}")
```

Expected labels: positive, negative, negative, positive. This simple code only flips a word when the immediately preceding token is a negator. It can fail on 'not at all good', sarcasm such as 'great, another day of smoke', and complex double negatives.

Alternative summary: selecting the first k sentences is O(1) only if sentence spans are already available; a practical implementation still scans/splits text in O(words). Frequency-based extraction is typically O(words) time and O(v+s) memory for vocabulary size v and sentence count s. First-k is fast but may miss the main point; frequency scoring may be more relevant but can overvalue repeated terms.

Final Submission Checklist

- Create CSA08-<RegisterNumber>-Assignment5 and ensure faculty access.
- Include Part A designs and SIMATS-style pseudocode; Python code does not replace required pseudocode.
- Include source files, datasets or data generator, SQLite creation script/database, charts, timing results and cProfile output.
- Run completed TODOs and capture genuine output screenshots.
- Add README.md with name, register number, Python version, library versions, commands and results.
- Add requirements.txt with actual installed versions of pandas/matplotlib if used.
- Commit progressively at least once per task and add the working GitHub link.
- Write interpretations from your own charts, performance measurements and 50-complaint review.

Suggested README.md Template

```
# CSA08 – Assignment 5 (CO5)
- Name: YOUR NAME
- Register Number: YOUR REGISTER NUMBER
- Python version: record `python --version`
- Libraries: record installed pandas/matplotlib versions

Workflow: Acquire → Clean → Store → Query → Visualise → Profile → Challenge → Interpret

Task folders: TaskA1, TaskA2, TaskA3, TaskA4, TaskA5, TaskB1, TaskB2, TaskB3, TaskB4

For each task, document the exact run command, result, and links to actual evidence.
```